

Tecnológico Nacional de México
Instituto Tecnológico de Mazatlán
Ingeniería en Sistemas Computacionales

Reporte de práctica

Semáforo peatonal con botón

Arduino Uno R4 WiFi · 5 LEDs · Pushbutton con antirrebote

Dato	Detalle
Materia	Sistemas Programables
Práctica	Semáforo peatonal con botón
Número de control	231000103
Placa	Arduino Uno R4 WiFi
Herramientas	Arduino IDE 2, Tinkercad Circuits
Fecha de pruebas	15 de septiembre de 2026
Fecha de entrega	16 de septiembre de 2026

Resumen

Se implementó un semáforo peatonal con Arduino Uno R4 WiFi: un semáforo de autos que cambia solo, en un ciclo repetido de verde (6000 ms), amarillo (2000 ms) y rojo (6000 ms), y un semáforo peatonal que permanece en rojo salvo que un botón haya solicitado el cruce durante el verde o el amarillo de los autos. Toda la temporización se implementó con `millis()`, sin usar `delay()`, y la lectura del botón incluye antirrebote por software. Las pruebas en el armado físico confirmaron que el semáforo de autos nunca se detiene y que el cruce peatonal solo se habilita cuando los autos están en rojo.

I. Introducción

Un semáforo con cruce peatonal es un ejemplo clásico de máquina de estados: el sistema debe recordar en qué fase se encuentra (verde, amarillo o rojo) y decidir cuándo pasar a la siguiente sin depender de que un humano lo controle paso a paso. A esto se agrega la necesidad de atender un evento externo —la solicitud de un peatón— sin que interrumpa el ciclo de los autos ni ponga en riesgo a nadie. Esta práctica combina una máquina de estados no bloqueante, construida con `millis()` [1], con la lectura de un botón con antirrebote por software [4], para que la solicitud de cruce solo se atienda en el momento seguro del ciclo.

II. Objetivos

Objetivo general. Controlar un semáforo de autos y un semáforo peatonal con Arduino, de modo que el cruce peatonal solo se habilite cuando el operador (peatón) lo solicita y sea seguro hacerlo.

- Implementar una máquina de estados no bloqueante (verde, amarillo, rojo) con `millis()`, con una duración distinta para cada estado.
- Leer un pushbutton con `INPUT_PULLUP` y aplicar antirrebote por software, sin usar `delay()`.
- Guardar la solicitud de cruce en una variable y atenderla únicamente cuando el semáforo de autos llega a rojo.
- Verificar que presionar el botón en un momento inseguro del ciclo (rojo de autos) no tenga ningún efecto.

III. Marco teórico

A. Máquinas de estado no bloqueantes con `millis()`

Un semáforo tiene, en todo momento, un estado bien definido (verde, amarillo o rojo) y reglas fijas para pasar de uno a otro. La forma no bloqueante de implementar esto en Arduino es guardar en una variable el momento (`millis()`) en que empezó el estado actual y, en cada vuelta del `loop()`, comparar cuánto tiempo ha pasado contra la duración de ese estado [1]; si se cumplió, se cambia de estado y se actualiza esa variable. A diferencia de usar `delay()`, el programa sigue revisando otras condiciones —como el botón— mientras espera a que termine cada estado.

B. Antirrebote (debounce) de botones

Un botón mecánico no cambia de HIGH a LOW de forma limpia: el contacto “rebota” durante unos milisegundos, lo que puede leerse como varias pulsaciones en vez de una sola [2], [4]. La técnica de antirrebote por software consiste en ignorar los cambios de lectura hasta que el valor se mantenga estable durante un tiempo mínimo (en esta práctica, 40 ms); solo entonces se acepta el nuevo estado del botón como válido. Usar `INPUT_PULLUP` [3] evita además la necesidad de una resistencia externa: el pin queda en HIGH por defecto y pasa a LOW al presionar el botón.

IV. Materiales y herramientas

Tabla I. Materiales y herramientas.

Elemento	Uso en la práctica
Arduino Uno R4 WiFi	Ejecuta la máquina de estados del semáforo y lee el botón
5 LEDs (2 verdes, 2 rojos, 1 amarillo)	Semáforo de autos (verde, amarillo, rojo) y semáforo peatonal (verde, rojo)
5 resistencias de 220 Ω	Limitan la corriente de cada LED
1 pushbutton	Permite solicitar el cruce peatonal
Protoboard	Permite armar el circuito sin soldar los componentes
Cables Dupont	Conexión entre el Arduino y la protoboard
Arduino IDE 2	Programación de la placa
Tinkercad Circuits	Elaboración del diagrama de conexiones

V. Desarrollo

A. Circuito

Los cinco LEDs (dos verdes, dos rojos y uno amarillo) se conectaron en la protoboard, cada uno en serie con una resistencia de 220 Ω , y el pushbutton se conectó entre el pin 2 y tierra, usando la resistencia de pull-up interna del Arduino (INPUT_PULLUP). La Figura 1 muestra el diagrama elaborado en Tinkercad Circuits, y la Tabla II detalla las conexiones.

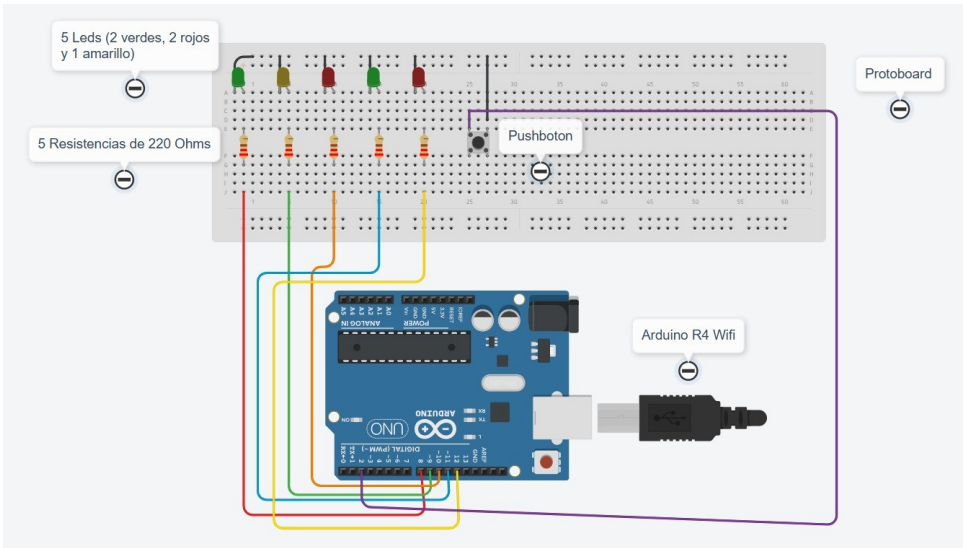


Figura 1. Diagrama de conexiones (Tinkercad Circuits).

Tabla II. Conexiones del circuito.

Elemento	Pin	Función
Rojo peatonal	Pin 12	Encendido casi todo el tiempo; se apaga al atender el

		cruce
Verde peatonal	Pin 11	Se enciende solo al atender una solicitud de cruce
Rojo autos	Pin 10	Encendido durante el estado “rojo” (6000 ms)
Amarillo autos	Pin 9	Encendido durante el estado “amarillo” (2000 ms)
Verde autos	Pin 8	Encendido durante el estado “verde” (6000 ms)
Pushbutton	Pin 2 (INPUT_PULLUP)	Solicita el cruce peatonal; presionado = LOW

B. Programa

Máquina de estados del semáforo de autos. El programa guarda en `inicioEstado` el momento en que empezó el estado actual (0 = verde, 1 = amarillo, 2 = rojo) y compara `millis()` contra ese valor y la duración del estado (6000, 2000 o 6000 ms) para decidir cuándo avanzar. Al llegar a rojo, el ciclo vuelve a verde y se repite indefinidamente, sin que el botón pueda alterar esta secuencia.

Lectura del botón y solicitud de cruce. En cada vuelta del `loop()` se lee el botón. Un cambio de lectura solo se acepta como válido si se mantuvo estable durante al menos 40 ms (antirrebote). Si el botón pasa a LOW mientras el semáforo de autos está en verde o en amarillo, se activa la bandera `solicitudPeaton`; presionarlo durante el rojo de autos no tiene ningún efecto, porque en ese momento la solicitud ya se resolvió o no puede resolverse de forma segura.

Atención de la solicitud. Cuando el semáforo de autos pasa de amarillo a rojo, el programa revisa `solicitudPeaton`: si estaba activa, enciende el verde peatonal y apaga el rojo peatonal; si no, el peatonal se queda en rojo. La bandera se reinicia a false de inmediato, y al volver los autos a verde, el peatonal regresa a rojo en cualquier caso.

VI. Resultados

El comportamiento del semáforo se determinó a partir del código fuente y se verificó grabando el circuito en video, presionando el botón en distintos momentos del ciclo; el video está disponible en [Video/Readme.txt](#) del repositorio de la práctica. La Figura 2 muestra el armado físico usado en las pruebas.

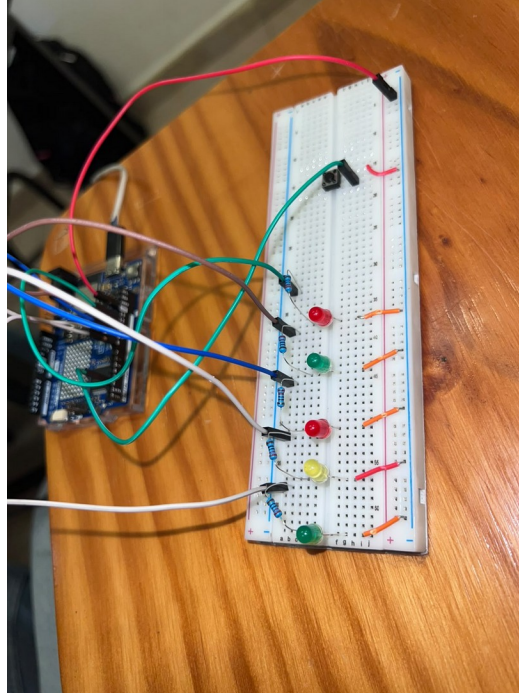


Figura 2. Armado físico con el Arduino Uno R4 WiFi, los 5 LEDs y el pushbutton.

Para comparar los dos comportamientos posibles del semáforo peatonal, se calculó, a partir de la lógica del programa, el estado de cada LED durante un ciclo completo de 14000 ms. La Figura 3 corresponde al caso sin solicitud de cruce y la Figura 4 al caso con solicitud durante el verde o el amarillo de los autos; la Tabla III resume ambos escenarios.

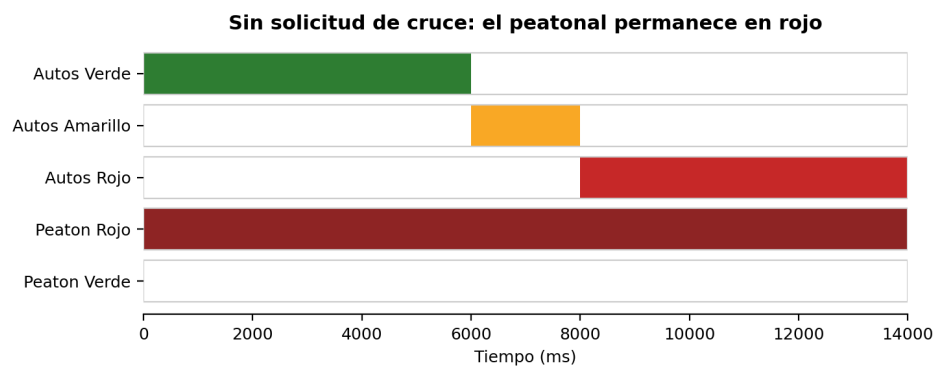


Figura 3. Sin solicitud de cruce: el semáforo peatonal permanece en rojo durante todo el ciclo.

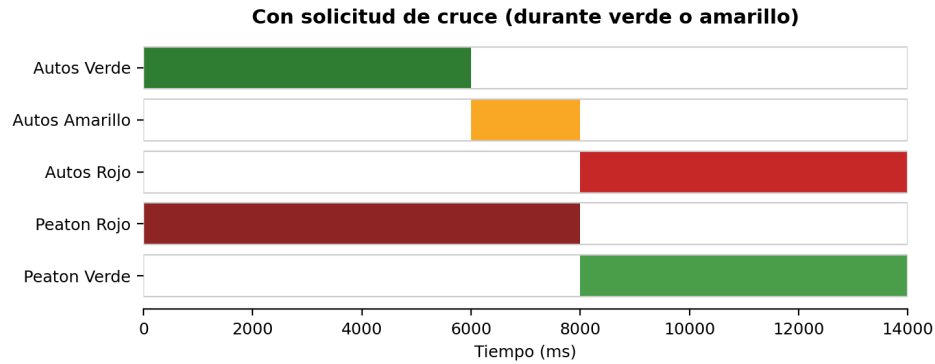


Figura 4. Con solicitud de cruce (durante verde o amarillo de autos): el peatonal se pone en verde exactamente cuando los autos llegan a rojo.

Tabla III. Estado del semáforo peatonal según el escenario.

Escenario	Semáforo peatonal
Sin solicitud de cruce	Rojo durante todo el ciclo (0–14000 ms)
Con solicitud durante verde o amarillo	Rojo de 0 a 8000 ms; verde de 8000 a 14000 ms
Solicitud presionada durante el rojo de autos	Sin efecto: la solicitud solo se guarda en verde o amarillo

Las gráficas completas y las observaciones detalladas se encuentran en Resultados/Resultados_semaforo.pdf.

VII. Análisis de resultados

Independencia del semáforo de autos. Como muestran las Figuras 3 y 4, el ciclo de 14000 ms del semáforo de autos (verde, amarillo, rojo) es idéntico en ambos escenarios: el botón nunca acelera, retrasa ni interrumpe esa secuencia. Esto confirma que la máquina de estados de los autos y la lógica del botón están desacopladas.

Ventana de cruce seguro. El semáforo peatonal solo cambia de comportamiento cuando la solicitud se hizo en la ventana correcta (verde o amarillo de autos). Presionar el botón durante el rojo de autos no tiene ningún efecto porque en ese momento ya no hay nada que decidir: o el cruce ya se habilitó al inicio del rojo, o no había forma de habilitarlo sin poner al semáforo de autos en un estado inconsistente.

Una solicitud, un cruce. Como solicitudPeaton se reinicia a false apenas se atiende, una sola pulsación del botón produce un único cruce peatonal por ciclo, sin importar cuántas veces se presione mientras la bandera ya está activa. Esto evita que el peatonal quede en verde más tiempo del debido.

Efecto del antirrebote. Durante las pruebas no se observaron activaciones falsas de solicitudPeaton por rebotes del botón, lo que es consistente con la ventana de 40 ms usada en el código; no se cuenta, sin embargo, con una medición instrumentada (osciloscopio) de los rebotes reales del botón para cuantificar ese margen con más precisión.

VIII. Conclusiones

Conclusión general. Se cumplió el objetivo de controlar un semáforo de autos y uno peatonal de forma coordinada y segura. La máquina de estados basada en millis() mantuvo el ciclo de los autos sin

interrupciones, mientras que la lógica de solicitudPeaton, combinada con el antirrebote del botón, permitió atender el cruce peatonal solo en el momento adecuado del ciclo, sin depender de delay() en ningún punto del programa.

Conclusión individual (L231000103). Esta práctica me ayudó a entender que una máquina de estados no es solo “encender y apagar LEDs en orden”, sino decidir en qué momentos el sistema puede aceptar una entrada externa sin ponerse en riesgo. Lo que más trabajo me costó fue separar la lectura del botón (que ocurre en cada vuelta del loop()) de la acción que provoca (que solo ocurre una vez, al cambiar a rojo). También noté la importancia del antirrebote: sin él, una sola pulsación podría registrarse varias veces. Como mejora, agregaría un LED o zumbador que avise al peatón cuánto tiempo le queda para cruzar antes de que el semáforo de autos vuelva a verde.

Referencias

- [1] Arduino, «millis()», Language Reference. Disponible en: <https://www.arduino.cc/reference/en/language/functions/time/millis/>
- [2] Arduino, «digitalRead()», Language Reference. Disponible en: <https://www.arduino.cc/reference/en/language/functions/digital-io/digitalread/>
- [3] Arduino, «pinMode() — INPUT_PULLUP», Language Reference. Disponible en: <https://www.arduino.cc/reference/en/language/functions/digital-io/pinmode/>
- [4] Arduino, «Debounce», ejemplo oficial. Disponible en: <https://docs.arduino.cc/built-in-examples/digital/Debounce/>
- [5] Arduino, «UNO R4 WiFi», documentación oficial. Disponible en: <https://docs.arduino.cc/hardware/uno-r4-wifi>